ECE 385

Spring 2025

Experiment #8

SOC with USB and VGA Interface in SystemVerilog
Lab-8

Cristina Natalia Lumban Tobing 3220300398
Artha Amanda Sidauruk 3220300399
TA: Qiyang Wu

Table of Contents

Experiment #8 1

Introduction 3

Written Description of Lab 8 System 3

Written Description of the entire Lab 8 System *3*

Written Description of the USB protocol *3*

Block Diagram *4*

Module Descriptions *4*

Answers to Hidden Questions 8

Answers to Post Lab Questions 8

Conclusion 9

Discussion of functionality of our design *9*

Difficulty in the lab and possible improvement *9*

Introduction

The primary objective of this laboratory assignment was to design and implement a System-on-Chip (SoC) capable of interfacing with both USB input devices and VGA output displays. This integration was accomplished on the DE2 development board using SystemVerilog and C programming for the NIOS II soft processor. The developed system allows a user to control the movement of a ball displayed on a VGA monitor using a USB keyboard. The directional movement is governed by the keys W, A, S, and D, and the ball is programmed to rebound upon contacting the display edges, simulating a simple game physics engine.

Written Description of Lab 8 System

Written Description of the entire Lab 8 System

In this experiment, we established a hardware-software co-design involving a USB keyboard input and VGA visual output. To realize this setup, a custom USB driver was implemented for handling communication with the onboard OTG (On-The-Go) controller. This included the ability to write to and read from specific USB buffer addresses. The driver structure required first specifying the register address, followed by executing either a write or read operation.

To support display functionality, a VGA driver was also developed and synchronized to a dedicated 25 MHz clock. This clock, operating at half the frequency of the DE2's default 50 MHz clock, ensured the screen refreshed near 60Hz, a necessary condition for flicker-free image rendering. The VGA logic managed both horizontal and vertical synchronization, crucial for accurate rendering of graphical elements.

Within the NIOS II environment, software routines were written to manipulate specific memory-mapped registers that controlled chip select lines, read/write flags, and data paths. For data transmission, the process required writing the target address to a specific register, then writing the data itself. For data retrieval, a similar process was followed where the address was written first, and the data read afterward.

Written Description of the USB protocol

- **IO_Write(address, data):** Directs a byte of data to a particular address on the OTG chip.
- **IO_Read(address):** Retrieves a byte of data from the given address.
- **USBWrite():** High-level abstraction that writes data from the NIOS II system to the OTG chip using sequential register-based communication.
- **USBRead():** Performs a read operation by issuing an address write followed by a data read from the corresponding register.

Block Diagram

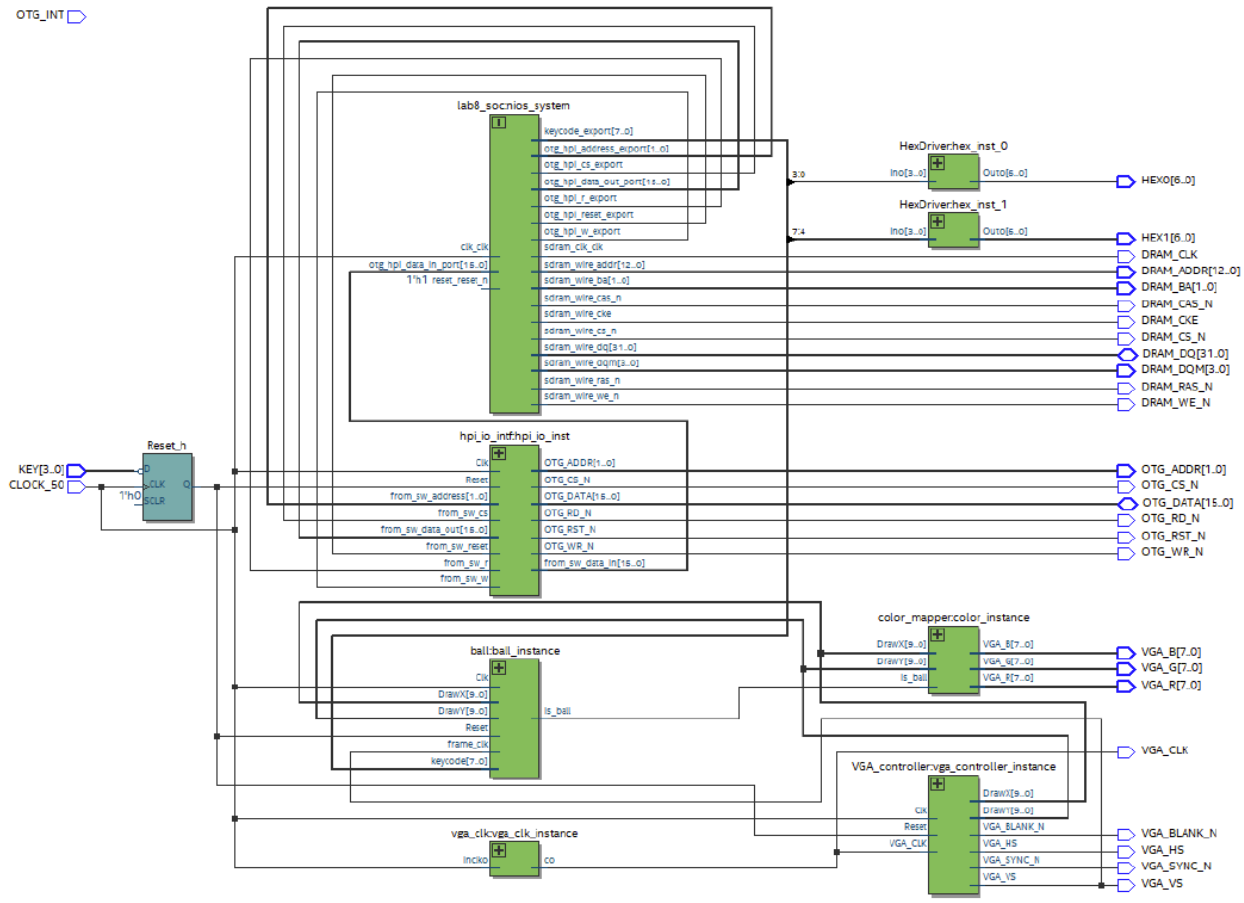


Figure 1 Top Level Block Diagram

Module Descriptions

Module: Ball
Inputs: Clk, Reset, frame_clk, keycode, DrawX, DrawY
Outputs: is_ball
Description: This module is responsible for rendering the ball and managing its movement based on keyboard input using SystemVerilog logic.
Purpose: It enables the ball to navigate in the directions: right, left, up, and down depending on key presses. When the ball reaches the screen edge, it automatically reverses direction to simulate a bounce.

Module: color mapper
Inputs: DrawX, DrawY, is ball
Outputs: VGA_R, VGA_G, VGA_B

Description: Determine the color to be displayed on the VGA monitor for each pixel
Purpose: The module evaluates whether the current pixel lies within the ball's boundary. If it does, the pixel is rendered white. If not, the background color is displayed, ensuring visual clarity between the moving ball and the static screen.

Module: HexDriver
Inputs: Input
Outputs: Output
Description: Displays the ASCII value of the key currently being pressed on the 7-segment HEX displays of the DE2 board.
Purpose: It receives the keycode and converts it into a suitable format for visual display on the hex display modules, allowing real-time feedback of keyboard input.

Module: Lab8
Inputs: CLOCK 50, KEY, OTG INT
Outputs: HEX0, HEX1, VGA_R, VGA_G, VGA_B, VGA_CLK, VGA_SYNC_N, VGA_BLANK_N, VGA_VS, VGA_HS, OTG_DATA, OTG_ADDR, OTG_CS_N, OTG_RD_N, OTG_WR_N, OTG_RST_N, OTG_INTDRAM_ADDR, DRAM_DQ, DRAM_BA, DRAM_DQM, DRAM_RAS_N, DRAM_CAS_N, DRAM_CKE, DRAM_WE_N, DRAM CS N, DRAM CLK
Description: Acts as the top-level integration module for the entire lab design.
Purpose: Links all individual submodules together. This includes interfacing between the NIOS II processor, the USB controller, and the VGA output system. It ensures synchronization and correct data flow between components.

Module: vga_clk
Inputs: inclk0
Outputs: c0
Description: Generates the clock signal used to drive the VGA controller.
Purpose: This module produces a 25MHz clock necessary for VGA operations, ensuring the screen is refreshed at a consistent 60Hz. It is calculated based on the total number of pixels (800 x 525) required for full-frame rendering.

Module: VGA_controller
Inputs: Clk, Reset, VGA_CLK
Outputs: VGA_HS, VGA_VS, VGA_BLANK_N, VGA_SYNC_N, DrawX, DrawY
Description: Translates the ball's position into visual data and manages the screen's timing signals.
Purpose: Manages the horizontal and vertical synchronization pulses needed for VGA output, and keeps track of which pixel is currently being drawn so the ball appears in the correct position.

Module: Hpi_io_intf
Inputs: Clk, Reset, from_sw_address, from_sw_data_out, from_sw_r, from_sw_w, from_sw_cs, from_sw_reset
Outputs: From_sw_data_in, OTG_ADDR, OTG_RD_N, OTG_WR_N, OTG_CS_N, OTG_RST_N, OTG_DATA
Description: Facilitates communication between the NIOS II processor and the USB (EZ-OTG) interface chip.
Purpose: This interface module is responsible for transmitting read/write commands, addresses, and data values to and from the OTG chip. It also manages the reset process for the USB controller and provides the necessary bus signals for successful data exchange.

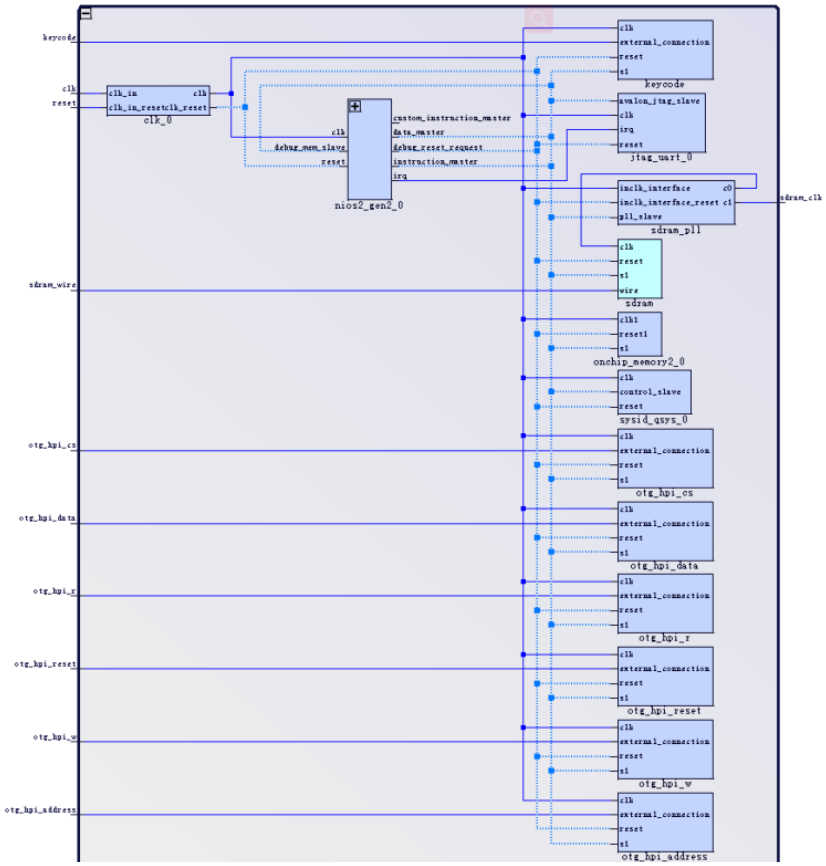
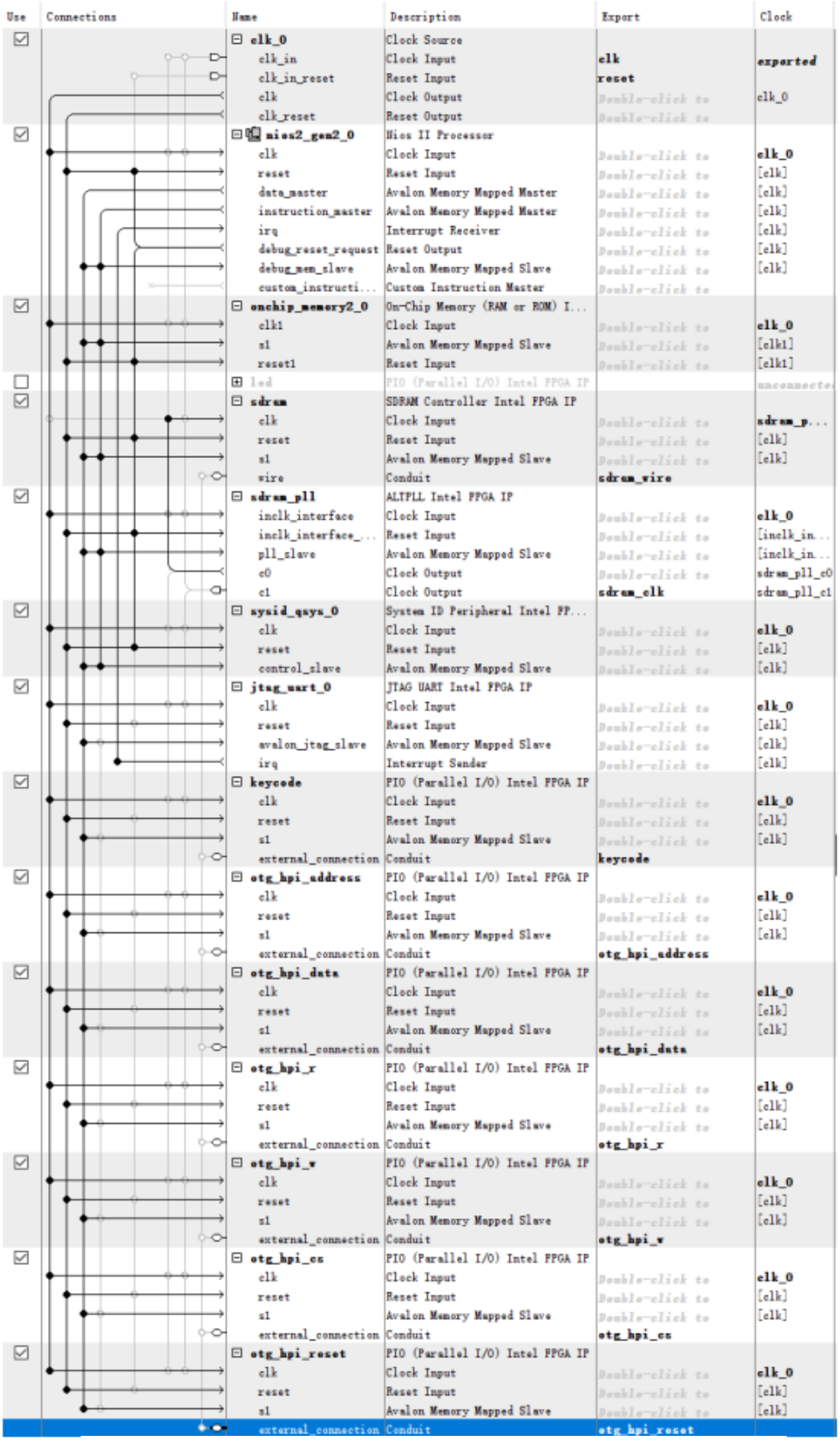


Figure 2: Platform Designer Schematic



Use	Connections	Name	Description	Export	Clock
☑		⊟ clk_0	Clock Source		
		clk_in	Clock Input	clk	exported
		clk_in_reset	Reset Input	reset	
		clk	Clock Output	Double-click to	clk_0
		clk_reset	Reset Output	Double-click to	
☑		⊟ nios2_gen2_0	Nios II Processor		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		data_master	Avalon Memory Mapped Master	Double-click to	[clk]
		instruction_master	Avalon Memory Mapped Master	Double-click to	[clk]
		irq	Interrupt Receiver	Double-click to	[clk]
		debug_reset_request	Reset Output	Double-click to	[clk]
		debug_mem_slave	Avalon Memory Mapped Slave	Double-click to	[clk]
		custom_instructi...	Custom Instruction Master	Double-click to	
☑		⊟ onchip_memory2_0	On-Chip Memory (RAM or ROM) I...		
		clk1	Clock Input	Double-click to	clk_0
		s1	Avalon Memory Mapped Slave	Double-click to	[clk1]
		reset1	Reset Input	Double-click to	[clk1]
☐		⊞ led	PIO (Parallel I/O) Intel FPGA IP		unconnecte
☑		⊟ sdram	SDRAM Controller Intel FPGA IP		
		clk	Clock Input	Double-click to	sdram_p...
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		wire	Conduit	sdram_wire	
☑		⊟ sdram_pll	ALTPLL Intel FPGA IP		
		inclk_interface	Clock Input	Double-click to	clk_0
		inclk_interface_...	Reset Input	Double-click to	[inclk_in...
		pll_slave	Avalon Memory Mapped Slave	Double-click to	[inclk_in...
		c0	Clock Output	Double-click to	sdram_pll_c0
		c1	Clock Output	sdram_clk	sdram_pll_c1
☑		⊟ sysid_qsys_0	System ID Peripheral Intel FP...		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		control_slave	Avalon Memory Mapped Slave	Double-click to	[clk]
☑		⊟ jtag_uart_0	JTAG UART Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to	[clk]
		irq	Interrupt Sender	Double-click to	[clk]
☑		⊟ keycode	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	keycode	
☑		⊟ otg_hpi_address	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	otg_hpi_address	
☑		⊟ otg_hpi_data	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	otg_hpi_data	
☑		⊟ otg_hpi_r	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	otg_hpi_r	
☑		⊟ otg_hpi_w	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	otg_hpi_w	
☑		⊟ otg_hpi_cs	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	otg_hpi_cs	
☑		⊟ otg_hpi_reset	PIO (Parallel I/O) Intel FPGA IP		
		clk	Clock Input	Double-click to	clk_0
		reset	Reset Input	Double-click to	[clk]
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]
		external_connection	Conduit	otg_hpi_reset	

Figure 3 Platform Designer View

Answers to Hidden Questions

1. **What are the advantages and/or disadvantages of using a USB interface over PS/2 interface to connect to the keyboard? List any two.**

- USB devices are hot-pluggable and immediately usable, unlike PS/2 which often requires rebooting.
- USB polling rate is significantly faster than that of PS/2, enhancing responsiveness.

2. **3.2 Notice that Ball Y Pos is updated using Ball Y Motion. Will the new value of Ball Y Motion be used when Ball Y Pos is updated, or the old? What is the difference between writing "Ball Y Pos in = Ball Y Pos + Ball Y Motion;" and "Ball Y Pos in = Ball Y Pos + Ball Y Motion in;"? How will this impact behavior of the ball during a bounce, and how might that interact with a response to a keypress?** When "Ball Y Pos" is updated, it typically uses the previous clock cycle's value of "Ball Y Motion" due to the nature of hardware parallelism. Writing "Ball Y Pos" = "Ball Y Pos" + "Ball Y Motion in" causes the position to reflect immediate changes like wall collisions or keypresses in the current cycle. This offers a quicker and more accurate simulation of bounce and control..

Answers to Post Lab Questions

1. **What is the difference between VGA_clk and Clk?**

- The system clock (Clk) at 50 MHz drives the logic and general system operations.
- The VGA clock (VGA_Clk) at 25 MHz ensures correct refresh timing on the VGA screen.

2. **In the file io_handler.h, why is it that the otg_hpi_data is defined as an integer pointer while the otg_hpi_r is defined as a char pointer?**
   - The otg_hpi_data is an int* because it deals with 32-bit data exchanges.
   - The otg_hpi_r is a char* since it only represents control bits or flags, which can be stored in a single byte.

3. **Document the Design Resources and Statistics in following table.**

LUT	2702
DSP	0
Memory (BRAM)	11390 bits
Flip-Flop	2264
Frequency	66.41MHz
Static Power	105.33mW
Dynamic Power	28.66mW
Total Power	180.45mW

Conclusion

Discussion of functionality of our design

The final system worked as intended. The ball moved responsively to keyboard inputs and rebounded accurately at the screen boundaries. Early challenges involved understanding the HPI protocol and integrating USB communication, but these were eventually resolved. The system showed no signs of timing violations or functional glitches during testing.

Difficulty in the lab and possible improvement

While the experiment was complex, it was suitable for an undergraduate course level. The guidance provided was generally sufficient, but more detailed documentation on the HPI interface would help reduce the initial learning curve. We recommend enhancing the lab manual with expanded explanations or flow diagrams of the USB data transaction cycle.